

Week 01: การเรียนรู้ Conditional Logic (if-else, switch-case) สำหรับ Game Development

ภาพรวมของ Assignment

Assignment นี้จะพาไปฝึกเขียน **conditional statements** (คำสั่งเงื่อนไข) และ **decision-making logic** (การตัดสินใจในโปรแกรม) ผ่านการเขียนโค้ดจริงทั้งหมด 11 methods โดยเน้นที่ if-else, switch-case และการรวมหลายเงื่อนไขเข้าด้วยกัน เพื่อจำลองสถานการณ์ที่พบได้จริงในเกม

วิธีดู "คำตอบ" ของแต่ละ method คือให้สั่งพิมพ์ผลลัพธ์ด้วย `Debug.Log()` แล้วเทียบกับผลลัพธ์ที่ตัวอย่าง Test Cases กำหนดไว้ในเอกสารนี้ ข้อความต้องตรงกันทุกตัวอักษร (รวมตัวพิมพ์เล็ก-ใหญ่และเว้นวรรค) ถึงจะถือว่าถูกต้อง

จุดประสงค์การเรียนรู้

หลังเรียนจบ assignment นี้ นักศึกษาจะสามารถ:

- เขียนโครงสร้าง if-else และ switch-case ได้อย่างถูกต้อง
- ใช้ AND (&&) และ OR (||) เพื่อรวมหลายเงื่อนไขเข้าด้วยกัน
- ตรวจสอบ edge case (ค่าขอบเขต เช่น ค่าติดลบ หรือค่าที่ไม่ถูกต้อง) ได้อย่างรอบคอบ
- นำ conditional logic ไปใช้แก้ปัญหาจริงในสถานการณ์แบบเกม
- เขียนโค้ดที่อ่านง่าย มีการจัดรูปแบบเรียบร้อย

โครงสร้างของ Assignment

งานแบ่งออกเป็น 3 ส่วน อยู่ใน 2 ไฟล์สคริปต์:

- **Example Methods (7 methods, ไฟล์ `Workshop.cs`)** — แบบฝึกหัดพื้นฐานที่ทำร่วมกันในห้องเรียน ไม่มีคะแนน ใช้ชื่อ `As01-As07`
- **Level 1: Simple (7 methods, ไฟล์ `Assignment.cs`)** — โจทย์ if-else และ switch-case ระดับพื้นฐาน ใช้ชื่อ `As01-As07`
- **Level 2: Moderate (4 methods, ไฟล์ `Assignment.cs`)** — โจทย์ conditional logic ที่ซับซ้อนขึ้น ผสมกับสถานการณ์ในเกม ใช้ชื่อ `As08-As11`

 ชื่อ `As01-As07` ใน `Workshop.cs` และ `As01-As07` ใน `Assignment.cs` เป็นคนละกลุ่มกัน (อยู่คนละไฟล์คนละคลาส) อย่าสับสนว่าเป็น method เดียวกัน

วิธีส่งค่า input เข้า method: แต่ละ method จะไม่มี parameter ให้ใส่ในวงเล็บแล้ว แต่จะรับค่าผ่าน **public field** ที่ประกาศไว้เหนือ method นั้น ๆ แทน ถ้าเปิดไฟล์ใน Unity แล้วลาก script ไปใส่ GameObject ก็จะมี field เหล่านี้ปรากฏใน Inspector ให้กรอกค่าได้โดยตรง ส่วนตอนรัน test cases ระบบจะกำหนดค่าลง field ให้เองตามที่ระบุใน Test Cases ของแต่ละ method

Example Methods (`Workshop.cs`)

Methods เหล่านี้แสดงแนวคิด conditional พื้นฐาน Implement เพื่อฝึกหัดแต่จะไม่มีภาระให้คะแนน

1. `As01_SyntaxIf` (2 test cases)

วัตถุประสงค์: แสดงความเข้าใจโครงสร้างของ if โดยเข้าใจว่าคำสั่งที่อยู่ นอก block ของ if จะทำงานเสมอ ไม่ว่าเงื่อนไขจะเป็นจริงหรือเท็จ

เนื้อเรื่อง: ประตูปราสาทเปิดเองเฉพาะตอนหกโมง (`isSixOClock` เป็นจริง) แต่ไม่ว่าประตูจะเปิดหรือไม่ ผู้เล่นก็เคาะประตูทุกครั้ง

Method Signature:

```
public bool isSixOClock;  
void As01_SyntaxIf()
```

Logic ที่ต้อง implement:

- ตรวจสอบว่า `isSixOClock` เป็นจริงหรือไม่
- ถ้าเป็นจริง ให้แสดงข้อความ "The door opens."
- แสดงข้อความ "Knock knock!" เสมอ ไม่ว่าเงื่อนไขจะเป็นจริงหรือเท็จ

Test Cases:

```
WS01.01: Input: true → Output:  
The door opens.  
Knock knock!  
WS01.01b: Input: false → Output:  
Knock knock!
```

Game Context: กลไกจับเวลา เช่น ประตูหรือกับดักที่เปิดเฉพาะช่วงเวลาที่กำหนด

Implementation Hint:

```
if (isSixOClock)  
{  
    Debug.Log("The door opens.");  
}  
Debug.Log("Knock knock!");
```

2. As02_StringComparisonExample (2 test cases)

วัตถุประสงค์: แสดงการเปรียบเทียบ string โดยใช้ if statements (`==`, `!=`)

Method Signature:

```
public string password;  
void As02_StringComparisonExample()
```

Logic ที่ต้อง implement:

- ตรวจสอบว่า password เท่ากับ "Moon" หรือไม่
- แสดงข้อความที่เหมาะสมสำหรับ password ที่ถูกต้อง/ผิด ("password is correct" หรือ "wrong password")

Test Cases:

```
WS01.02: Input: "Moon" → Output:  
password is correct  
WS01.02b: Input: "Sun" → Output:  
wrong password
```

Game Context: ระบบตรวจสอบ password, การ authentication ผู้ใช้

Implementation Hint:

```
if (password != "Moon")  
{  
    Debug.Log("wrong password");  
}  
if (password == "Moon")  
{  
    Debug.Log("password is correct");  
}
```

3. As03_NumberComparisonExample (3 test cases)

วัตถุประสงค์: แสดงการเปรียบเทียบตัวเลขโดยใช้ if statements

Method Signature:

```
public int as03Number;  
void As03_NumberComparisonExample()
```

Logic ที่ต้อง implement:

- เปรียบเทียบ as03Number กับ 10 โดยใช้ comparison operators ทั้งหมด
- โดยที่ลำดับการเขียนตรวจสอบแต่ละเครื่องหมายจะเรียงลำดับดังนี้ >, <, ==, >=, <=, != แสดงผลลัพธ์สำหรับการเปรียบเทียบแต่ละตัวที่เป็น true

Test Cases:

```
WS01.03: Input: 11 → Output:  
My Number > 10  
My Number >= 10  
My Number != 10  
WS01.03b: Input: 10 → Output:  
My Number == 10
```

```

My Number >= 10
My Number <= 10
WS01.03c: Input: 9 → Output:
My Number < 10
My Number <= 10
My Number != 10

```

Game Context: การเปรียบเทียบคะแนน, ข้อกำหนดระดับ, การตรวจสอบ stats

Implementation Hint:

```

if (as03Number > 10) // "My Number > 10"
if (as03Number < 10) // "My Number < 10"
if (as03Number == 10) // "My Number == 10"
if (as03Number >= 10) // "My Number >= 10"
if (as03Number <= 10) // "My Number <= 10"
if (as03Number != 10) // "My Number != 10"

```

4. As04_AndOrOperatorExample (3 test cases)

วัตถุประสงค์: แสดง AND และ OR operators ใน if statements (&&, ||)

Method Signature:

```

public int as04Number;
void As04_AndOrOperatorExample()

```

Logic ที่ต้อง implement:

- ตรวจสอบว่า as04Number อยู่ระหว่าง 8 และ 12 (exclusive) โดยใช้ AND
- ตรวจสอบว่า as04Number ตรงกับเงื่อนไข OR (> 8 OR < 12)
- แสดงข้อความที่เหมาะสม

Test Cases:

```

WS01.04: Input: 10 → Output:
My Number 8 > < 12
My Number or 8 || 12
WS01.04b: Input: 7 → Output:
My Number or 8 || 12
WS01.04c: Input: 13 → Output:
My Number or 8 || 12

```

Game Context: การตรวจสอบช่วง, conditional game mechanics

Implementation Hint:

```
if (as04Number > 8 && as04Number < 12) // "My Number 8 > < 12"  
if (as04Number > 8 || as04Number < 12) // "My Number or 8 || 12"
```

5. As05_GuessingNumberExample (2 test cases)

วัตถุประสงค์: เกมทายตัวเลขง่ายๆ โดยใช้ if-else statements โดย as05RandomNumber จะถูกกำหนดจาก Test case

Method Signature:

```
public int as05GuessingNumber;  
public int as05RandomNumber;  
void As05_GuessingNumberExample()
```

Logic ที่ต้อง implement:

- แสดงตัวเลขเป้าหมายด้วย `Debug.Log($"Guessing number {as05RandomNumber}");` เสมอ
- เปรียบเทียบ as05GuessingNumber กับเป้าหมาย as05RandomNumber
 - ชนะ : แสดงข้อความ `Congratulations! You guessed the correct number.`
 - แพ้ : แสดงข้อความ `I guess we can just agree to disagree.`

Test Cases:

```
WS01.05: Input: 5, 5 → Output:  
Guessing number 5  
Congratulations! You guessed the correct number.  
WS01.05b: Input: 3, 5 → Output:  
Guessing number 5  
I guess we can just agree to disagree.
```

Game Context: Mini-games, random events, mechanics ที่ขึ้นอยู่กับโชค

6. As06_GuessingNumberMoreOrLessExample (3 test cases)

วัตถุประสงค์: เกมทายตัวเลขขั้นสูงพร้อม hints ทิศทางโดยใช้ if-else-if โดย as06RandomNumber จะถูกกำหนดจาก Test case

Method Signature:

```
public int as06GuessingNumber;  
public int as06RandomNumber;  
void As06_GuessingNumberMoreOrLessExample()
```

Logic ที่ต้อง implement:

- แสดงตัวเลขเป้าหมายด้วย `Debug.Log($"Guessing number {as06RandomNumber}");` เสมอ

- จากนั้นให้ feedback ตามเปรียบเทียบ as06GuessingNumber กับ as06RandomNumber:
 - ต่ำเกินไป: Too low! Try again.
 - สูงเกินไป: Too high! Try again.
 - ถูกต้อง: Congratulations! We are same mind.

Test Cases:

```
WS01.06: Input: 3, 5 → Output:
Guessing number 5
Too low! Try again.
WS01.06b: Input: 7, 5 → Output:
Guessing number 5
Too high! Try again.
WS01.06c: Input: 5, 5 → Output:
Guessing number 5
Congratulations! We are same mind.
```

Game Context: Puzzle games, ระบบปรับความยาก

7. As07_VerifyIdentityExample (4 test cases)

วัตถุประสงค์: การตรวจสอบตัวตนหลายระดับโดยใช้ nested if statements

Method Signature:

```
public string as07Username;
public string as07Password;
public int as07Age;
public bool as07IsPaid;
void As07_VerifyIdentityExample()
```

Logic ที่ต้อง implement:

- ตรวจสอบ as07Username และ as07Password (username == "user", password == "user123")
 - ถ้า username และ password ถูกต้อง ให้แสดง "You have user access." แล้วทำการตรวจสอบเงื่อนไขย่อยซ้อนด้านใน (nested) ต่อไปนี้:
 - ตรวจสอบสถานะ VIP:
 - ถ้า as07IsPaid เป็นจริง แสดงข้อความ "welcome vip member."
 - ถ้า as07IsPaid เป็นเท็จ แสดงข้อความ "welcome free member."
 - ตรวจสอบอายุ:
 - ถ้า as07Age มากกว่า 18 แสดงข้อความ "You have access to exclusive content."
 - ถ้า username หรือ password ไม่ถูกต้อง ให้แสดงข้อความ "You have guest access." (และไม่ต้องตรวจสอบ VIP หรืออายุต่อ)

Test Cases:

WS01.07: Input: "user", "user123", 20, true → Output:
You have user access.
welcome vip member.
You have access to exclusive content.

WS01.07b: Input: "user", "user123", 15, true → Output:
You have user access.
welcome vip member.

WS01.07c: Input: "user", "user123", 20, false → Output:
You have user access.
welcome free member.

WS01.07d: Input: "guest", "pass", 20, false → Output:
You have guest access.

Game Context: การ authentication ผู้ใช้, การเข้าถึงเนื้อหา premium, การตรวจสอบอายุ

Level 1: Simple (Assignment.cs)

1. As01_CheckNumberSign (5 test cases)

วัตถุประสงค์: กำหนดว่าตัวเลขเป็นบวก ลบ หรือศูนย์

Method Signature:

```
public int as01Number;  
void As01_CheckNumberSign()
```

Logic ที่ต้อง implement:

- ใช้ if-else-if chain เพื่อตรวจสอบเครื่องหมายของ as01Number
- แสดง "Positive", "Negative" หรือ "Zero"

Test Cases:

AS01.01: Input: 5 → Output: Positive
AS01.01b: Input: -3 → Output: Negative
AS01.01c: Input: 0 → Output: Zero
AS01.01d: Input: 2147483647 → Output: Positive
AS01.01e: Input: -2147483648 → Output: Negative

Game Context: การคำนวณพลังชีวิต/ความเสียหาย, การตรวจสอบคะแนน, การตรวจจับทิศทาง

Implementation Hint:

```
// if (as01Number > 0)
//   Debug.Log("Positive");
// else if (as01Number < 0)
//   Debug.Log("Negative");
// else
//   Debug.Log("Zero");
```

2. As02_GetDayName (10 test cases)

วัตถุประสงค์: คำนวณชื่อวันสำหรับจำนวนเต็มที่กำหนด

Method Signature:

```
public int as02Day;
void As02_GetDayName()
```

Logic ที่ต้อง implement:

- ใช้ if-else-if หรือ switch-case เพื่อแมป as02Day กับชื่อวัน
- จัดการ input ที่ไม่ถูกต้องด้วย "Invalid day"
- โดยกำหนดให้ 1=Monday, 2=Tuesday, 3=Wednesday, 4=Thursday, 5=Friday, 6=Saturday, 7=Sunday, other=Invalid day

Test Cases:

```
AS01.02: Input: 1 → Output: Monday
AS01.02b: Input: 2 → Output: Tuesday
AS01.02c: Input: 3 → Output: Wednesday
AS01.02d: Input: 4 → Output: Thursday
AS01.02e: Input: 5 → Output: Friday
AS01.02f: Input: 6 → Output: Saturday
AS01.02g: Input: 7 → Output: Sunday
AS01.02h: Input: 0 → Output: Invalid day
AS01.02i: Input: 8 → Output: Invalid day
AS01.02j: Input: -5 → Output: Invalid day
```

Game Context: ระบบ quest รายวัน, calendar events, mechanics ที่ขึ้นอยู่กับเวลา

Implementation Hint:

```
// switch (as02Day)
// {
//   case 1: Debug.Log("Monday"); break;
//   case 2: Debug.Log("Tuesday"); break;
//   ...
```



```
// default: Debug.Log("Invalid day"); break;  
// }
```

3. As03_ValidatePassword (7 test cases)

วัตถุประสงค์: ตรวจสอบ password input ด้วยการจับคู่ string

Method Signature:

```
public string as03InputPassword;  
public string as03CorrectPassword;  
void As03_ValidatePassword()
```

Logic ที่ต้อง implement:

- เปรียบเทียบ strings แบบแม่นยำ (case-sensitive)
- แสดง "True" หรือ "False"
- โดย testcase กำหนดให้ as03CorrectPassword มีค่า "secret123"

Test Cases:

```
AS01.03: Input: "secret123", "secret123" → Output: True  
AS01.03b: Input: "wrongpass", "secret123" → Output: False  
AS01.03c: Input: "", "" → Output: True  
AS01.03d: Input: "secret123", "Secret123" → Output: False  
AS01.03e: Input: " secret123 ", "secret123" → Output: False  
AS01.03f: Input: "secret123", "" → Output: False  
AS01.03g: Input: "", "secret123" → Output: False
```

Game Context: ระบบ login, พื้นที่ปลอดภัย, การตรวจสอบ cheat code

Implementation Hint:

```
// if (as03InputPassword == as03CorrectPassword)  
//   Debug.Log("True");  
// else  
//   Debug.Log("False");
```

4. As04_GetGrade (14 test cases)

วัตถุประสงค์: คำนวณเกรดตัวอักษรสำหรับคะแนนที่กำหนด

Method Signature:

```
public int as04Score;  
void As04_GetGrade()
```

Logic ที่ต้อง implement:

- ใช้ if-else chain กับ as04Score thresholds
- A:80, B:70, C:60, D:50, F: อื่นๆ

Test Cases:

```
AS01.04: Input: 95 → Output: A  
AS01.04b: Input: 85 → Output: A  
AS01.04c: Input: 80 → Output: A  
AS01.04d: Input: 75 → Output: B  
AS01.04e: Input: 70 → Output: B  
AS01.04f: Input: 65 → Output: C  
AS01.04g: Input: 60 → Output: C  
AS01.04h: Input: 55 → Output: D  
AS01.04i: Input: 50 → Output: D  
AS01.04j: Input: 49 → Output: F  
AS01.04k: Input: 0 → Output: F  
AS01.04l: Input: 100 → Output: A  
AS01.04m: Input: -1 → Output: F  
AS01.04n: Input: 101 → Output: A
```

Game Context: ระบบจัดอันดับผู้เล่น, ระดับ achievement, การประเมินผลงาน

Implementation Hint:

```
// if (as04Score >= 80)  
//     Debug.Log("A");  
// else if (as04Score >= 70)  
//     Debug.Log("B");  
// ...
```

5. As05_IsLeapYear (9 test cases)

วัตถุประสงค์: ตรวจสอบว่าปีเป็นปีอธิกสุรทินหรือไม่โดยใช้กฎที่ซับซ้อน

Method Signature:

```
public int as05Year;  
void As05_IsLeapYear()
```

Logic ที่ต้อง implement:

- หารด้วย 400 ลงตัว: ป็อธิกสุรทิน
- หารด้วย 100 ลงตัว (แต่ไม่หาร 400 ลงตัว): ไม่ใช่ป็อธิกสุรทิน
- หารด้วย 4 ลงตัว (แต่ไม่หาร 100 ลงตัว): ป็อธิกสุรทิน
- อื่นๆ: ไม่ใช่ป็อธิกสุรทิน

Test Cases:

```
AS01.05: Input: 2000 → Output: True
AS01.05b: Input: 2020 → Output: True
AS01.05c: Input: 1900 → Output: False
AS01.05d: Input: 2004 → Output: True
AS01.05e: Input: 2100 → Output: False
AS01.05f: Input: 2400 → Output: True
AS01.05g: Input: 1999 → Output: False
AS01.05h: Input: -400 → Output: True
AS01.05i: Input: 0 → Output: True
```

Game Context: ระบบปฏิทิน, seasonal events, mechanics ที่ขึ้นอยู่กับเวลา

Implementation Hint:

```
// if (as05Year % 400 == 0)
//   Debug.Log("True");
// else if (as05Year % 100 == 0)
//   Debug.Log("False");
// else if (as05Year % 4 == 0)
//   Debug.Log("True");
// else
//   Debug.Log("False");
```

6. As06_Calculate (12 test cases)

วัตถุประสงค์: สร้างโปรแกรมเครื่องคิดเลขอย่างง่ายโดยตรวจสอบเครื่องหมายก่อนตัดสินใจดำเนินการพร้อมวิธีป้องกันการคำนวณผิดพลาด

Method Signature:

```
public double as06Num1;
public char as06Op;
public double as06Num2;
void As06_Calculate()
```

Logic ที่ต้อง implement:

- รองรับ operators +, -, *, /
- จัดการการหารด้วยศูนย์ จะต้องแสดงข้อความ **Error: Cannot divide by zero.**

- จัดรูปแบบตัวเลขโดยไม่มีทศนิยมที่ไม่จำเป็น
- ตรวจสอบ operator input

Test Cases:

```
AS01.06: Input: 5.0, '+', 3.0 → Output: Result: 8
AS01.06b: Input: 5.0, '-', 3.0 → Output: Result: 2
AS01.06c: Input: 5.0, '*', 3.0 → Output: Result: 15
AS01.06d: Input: 6.0, '/', 2.0 → Output: Result: 3
AS01.06e: Input: 6.0, '/', 0.0 → Output: Error: Cannot divide by zero.
AS01.06f: Input: 6.0, 'x', 2.0 → Output: Invalid operator. Please use +, -, *, or /.
AS01.06g: Input: -5.0, '+', -3.0 → Output: Result: -8
AS01.06h: Input: 0.0, '+', 0.0 → Output: Result: 0
AS01.06i: Input: 1e10, '+', 1e10 → Output: Result: 20000000000
AS01.06j: Input: 0.1, '+', 0.2 → Output: Result: 0.3
AS01.06k: Input: 5.0, ' ', 3.0 → Output: Invalid operator. Please use +, -, *, or /.
AS01.06l: Input: 5.0, 'X', 3.0 → Output: Invalid operator. Please use +, -, *, or /.
```

Game Context: การคำนวณความเสียหาย, การจัดการ resources, การคำนวณ stats

Implementation Tips:

- ใช้ switch-case สำหรับ operators
- จัดรูปแบบผลลัพธ์โดยใช้ string formatting ที่เหมาะสม

Implementation Hint:

```
// switch (as06Op)
// {
//   case '+': result = as06Num1 + as06Num2; break;
//   case '/':
//     if (as06Num2 == 0)
//       Debug.Log("Error: Cannot divide by zero.");
//     else
//       result = as06Num1 / as06Num2;
//     break;
//   default:
//     Debug.Log("Invalid operator. Please use +, -, *, or /.");
//     return;
// }
```

7. As07_GetSeason (16 test cases)

วัตถุประสงค์: คำนวณฤดูกาลสำหรับเดือนที่กำหนดพร้อมการตรวจสอบที่ครอบคลุม

Method Signature:

```
public int as07Month;  
void As07_GetSeason()
```

Logic ที่ต้อง implement:

- Winter: December (12), January (1), February (2)
- Spring: March (3) ถึง May (5)
- Summer: June (6) ถึง August (8)
- Fall: September (9) ถึง November (11)
- Invalid: เดือนนอกช่วง 1-12

Test Cases:

```
AS01.07: Input: 1 → Output: It's Winter.  
AS01.07b: Input: 2 → Output: It's Winter.  
AS01.07c: Input: 12 → Output: It's Winter.  
AS01.07d: Input: 3 → Output: It's Spring.  
AS01.07e: Input: 4 → Output: It's Spring.  
AS01.07f: Input: 5 → Output: It's Spring.  
AS01.07g: Input: 6 → Output: It's Summer.  
AS01.07h: Input: 7 → Output: It's Summer.  
AS01.07i: Input: 8 → Output: It's Summer.  
AS01.07j: Input: 9 → Output: It's Fall.  
AS01.07k: Input: 10 → Output: It's Fall.  
AS01.07l: Input: 11 → Output: It's Fall.  
AS01.07m: Input: 0 → Output: Invalid month number. Please enter a number between 1  
and 12.  
AS01.07n: Input: 13 → Output: Invalid month number. Please enter a number between  
1 and 12.  
AS01.07o: Input: -1 → Output: Invalid month number. Please enter a number between  
1 and 12.  
AS01.07p: Input: 100 → Output: Invalid month number. Please enter a number between  
1 and 12.
```

Game Context: Seasonal events, ระบบสภาพอากาศ, การเปลี่ยนแปลงสิ่งแวดล้อม

Implementation Hint:

```
// if (as07Month >= 1 && as07Month <= 12)  
// {  
//     if (as07Month == 12 || as07Month == 1 || as07Month == 2)  
//         Debug.Log("It's Winter.");  
//     else if (as07Month >= 3 && as07Month <= 5)  
//         Debug.Log("It's Spring.");  
//     ...  
// }
```

```
// else
// Debug.Log("Invalid month number. Please enter a number between 1 and 12.");
```

Level 2: Moderate (Assignment.cs)

1. As08_PurchasingSystemExample (4 test cases)

วัตถุประสงค์: ระบบซื้อขายโดยใช้ nested if statements

Method Signature:

```
public int as08Quantity;
public int as08Price;
public int as08Payment;
void As08_PurchasingSystemExample()
```

Logic ที่ต้อง implement:

- ตรวจสอบว่าสินค้าใน stock (`as08Quantity`) มีอยู่หรือไม่
 - ถ้า `as08Quantity <= 0` ให้แสดงข้อความ "สินค้าหมด"
 - ถ้ามีสินค้าใน stock (`as08Quantity > 0`) ให้ตรวจสอบเงินชำระ (`as08Payment`) กับราคาสินค้า 1 ชิ้น (`as08Price`):
 - ถ้า `as08Payment >= as08Price`:
 - แสดงข้อความ "คุณได้รับสินค้าแล้ว"
 - หากมีเงินทอน (`as08Payment > as08Price`) ให้แสดงข้อความ "คุณได้รับเงินทอน {as08Payment - as08Price} บาท"
 - ถ้า `as08Payment < as08Price`:
 - แสดงข้อความ "คุณมีเงินไม่พอ"

Test Cases:

```
AS01.08a: Input: 1, 10, 20 → Output:
คุณได้รับสินค้าแล้ว
คุณได้รับเงินทอน 10 บาท
AS01.08b: Input: 1, 10, 10 → Output:
คุณได้รับสินค้าแล้ว
AS01.08c: Input: 1, 10, 5 → Output:
คุณมีเงินไม่พอ
AS01.08d: Input: 0, 10, 20 → Output:
สินค้าหมด
```

Game Context: ร้านค้าในเกม, การจัดการ inventory, ระบบเศรษฐกิจ

2. As09_RockPaperScissorsExample (6 test cases)

วัตถุประสงค์: Logic เกมคลาสสิกโดยใช้ if-else statements

Method Signature:

```
public int as09UserChoice;
public int as09ComputerChoice;
void As09_RockPaperScissorsExample()
```

Logic ที่ต้อง implement:

- ตรวจสอบ as09UserChoice (0=Rock, 1=Paper, 2=Scissors)
 - หาก as09UserChoice ไม่อยู่ในช่วง 0 ถึง 2 (เช่น < 0 หรือ > 2) ให้แสดงข้อความ "กรุณาเลือกเป็นตัวเลขที่ถูกต้อง"
- หากตัวเลขถูกต้อง ให้เปรียบเทียบกับ as09ComputerChoice (0=Rock, 1=Paper, 2=Scissors) แล้วแสดงผลลัพธ์:
 - กรณีเสมอ (เลือกเหมือนกัน): "เสมอ"
 - กรณีผู้เล่นชนะ (0 ชนะ 2, 1 ชนะ 0, 2 ชนะ 1): "คุณชนะ!"
 - กรณีผู้เล่นแพ้: "คุณแพ้!"

Test Cases:

```
AS01.09a: Input: 0, 0 → Output: เสมอ
AS01.09b: Input: 0, 2 → Output: คุณชนะ!
AS01.09c: Input: 1, 0 → Output: คุณชนะ!
AS01.09d: Input: 2, 1 → Output: คุณชนะ!
AS01.09e: Input: 2, 0 → Output: คุณแพ้!
AS01.09f: Input: 3, 0 → Output: กรุณาเลือกเป็นตัวเลขที่ถูกต้อง
```

3. As10_CalculateWeaponDamage (6 test cases)

วัตถุประสงค์: คำนวณความเสียหายของอาวุธโดยใช้ multipliers ตามประเภท

Method Signature:

```
public string as10WeaponType;
public int as10BaseDamage;
void As10_CalculateWeaponDamage()
```

Logic ที่ต้อง implement:

- ใช้ damage multipliers ตามประเภทอาวุธ
- จัดการ weapon type input แบบไม่คำนึงถึงตัวใหญ่เล็ก
- แสดงความเสียหายสุดท้ายเป็น integer

Weapon Type Multipliers:

- sword: 1.3 (โบนัส 30%)
- axe: 1.4 (โบนัส 40%)

- bow: 1.2 (โบนัส 20%)
- staff: 1.5 (โบนัส 50%)
- dagger: 1.1 (โบนัส 10%)
- unknown/other: 1.0 (ไม่มีโบนัส)

Test Cases:

```
AS01.10a: Input: "sword", 50 → Output: 65
AS01.10b: Input: "axe", 50 → Output: 70
AS01.10c: Input: "bow", 50 → Output: 60
AS01.10d: Input: "staff", 50 → Output: 75
AS01.10e: Input: "dagger", 50 → Output: 55
AS01.10f: Input: "unknown", 50 → Output: 50
```

Game Context: ระบบการต่อสู้, การปรับสมดุลอาวุธ, การคำนวณความเสียหาย

Implementation Tips:

- ใช้ switch-case กับ toLowerCase() สำหรับการจัดการ case
- แปลงผลลัพธ์สุดท้ายเป็น integer

Implementation Hint:

```
// double multiplier = 1.0;
// switch (as10WeaponType?.ToLower())
// {
//     case "sword": multiplier = 1.3; break;
//     case "axe": multiplier = 1.4; break;
//     ...
//     default: multiplier = 1.0; break;
// }
// int totalDamage = (int)(as10BaseDamage * multiplier);
// Debug.Log(totalDamage.ToString());
```

4. As11_DeterminePlayerRank (33 test cases)

วัตถุประสงค์: กำหนดอันดับผู้เล่นและคำนวณรางวัลตามคะแนนและเวลาที่ใช้ในการเล่น

Method Signature:

```
public int as11Score;
public int as11CompletionTime;
void As11_DeterminePlayerRank()
```

Logic ที่ต้อง implement:

- ตรวจสอบ inputs (หาก `as11Score < 0` หรือ `as11CompletionTime < 0` ให้แสดง "Invalid score or time")
- กำหนดอันดับตาม score thresholds:
 - Gold: 8000+ คะแนน (100 base coins)
 - Silver: 6000-7999 คะแนน (75 base coins)
 - Bronze: 4000-5999 คะแนน (50 base coins)
 - Participation: 0-3999 คะแนน (25 base coins)
- คำนวณ Time Bonus:
 - 0-30 นาที: +25 coins
 - 31-60 นาที: +10 coins
 - 61+ นาที: +0 coins
- แสดงข้อความอันดับและรางวัลที่จัดรูปแบบ: `{rank} Rank - {baseCoins + timeBonus} coins earned!`

Test Cases:

```

AS01.11a: Input: -1, 30 → Output: Invalid score or time
AS01.11b: Input: 5000, -5 → Output: Invalid score or time
AS01.11c: Input: -10, -10 → Output: Invalid score or time
AS01.11d: Input: 0, 0 → Output: Participation Rank - 50 coins earned!
AS01.11e: Input: 0, 30 → Output: Participation Rank - 50 coins earned!
AS01.11f: Input: 0, 31 → Output: Participation Rank - 35 coins earned!
AS01.11g: Input: 0, 60 → Output: Participation Rank - 35 coins earned!
AS01.11h: Input: 0, 61 → Output: Participation Rank - 25 coins earned!
AS01.11i: Input: 2000, 25 → Output: Participation Rank - 50 coins earned!
AS01.11j: Input: 2000, 35 → Output: Participation Rank - 35 coins earned!
AS01.11k: Input: 2000, 70 → Output: Participation Rank - 25 coins earned!
AS01.11l: Input: 3999, 30 → Output: Participation Rank - 50 coins earned!
AS01.11m: Input: 3999, 60 → Output: Participation Rank - 35 coins earned!
AS01.11n: Input: 3999, 61 → Output: Participation Rank - 25 coins earned!
AS01.11o: Input: 4000, 0 → Output: Bronze Rank - 75 coins earned!
AS01.11p: Input: 4000, 30 → Output: Bronze Rank - 75 coins earned!
AS01.11q: Input: 4000, 31 → Output: Bronze Rank - 60 coins earned!
AS01.11r: Input: 4000, 60 → Output: Bronze Rank - 60 coins earned!
AS01.11s: Input: 4000, 61 → Output: Bronze Rank - 50 coins earned!
AS01.11t: Input: 4500, 45 → Output: Bronze Rank - 60 coins earned!
AS01.11u: Input: 5999, 30 → Output: Bronze Rank - 75 coins earned!
AS01.11v: Input: 5999, 60 → Output: Bronze Rank - 60 coins earned!
AS01.11w: Input: 5999, 61 → Output: Bronze Rank - 50 coins earned!
AS01.11x: Input: 6000, 0 → Output: Silver Rank - 100 coins earned!
AS01.11y: Input: 6000, 30 → Output: Silver Rank - 100 coins earned!
AS01.11z: Input: 6000, 31 → Output: Silver Rank - 85 coins earned!
AS01.11aa: Input: 6000, 60 → Output: Silver Rank - 85 coins earned!
AS01.11ab: Input: 6000, 61 → Output: Silver Rank - 75 coins earned!
AS01.11ac: Input: 6500, 45 → Output: Silver Rank - 85 coins earned!
AS01.11ad: Input: 7999, 30 → Output: Silver Rank - 100 coins earned!
AS01.11ae: Input: 8000, 30 → Output: Gold Rank - 125 coins earned!
AS01.11af: Input: 8000, 60 → Output: Gold Rank - 110 coins earned!
AS01.11ag: Input: 8500, 61 → Output: Gold Rank - 100 coins earned!

```

Implementation Tips:

- ตรวจสอบ inputs ก่อน
- ใช้ if-else chain สำหรับการกำหนดอันดับ
- ใช้ nested if-else สำหรับการคำนวณ time bonus

Implementation Hint:

```
// if (as11Score < 0 || as11CompletionTime < 0)
// {
//     Debug.Log("Invalid score or time");
//     return;
// }
//
// string rank; int baseCoins;
// if (as11Score >= 8000) { rank = "Gold"; baseCoins = 100; }
// else if (as11Score >= 6000) { rank = "Silver"; baseCoins = 75; }
// ...
//
// int timeBonus = 0;
// if (as11CompletionTime <= 30) timeBonus = 25;
// else if (as11CompletionTime <= 60) timeBonus = 10;
//
// int totalCoins = baseCoins + timeBonus;
// Debug.Log($"{rank} Rank - {totalCoins} coins earned!");
```

⚠ ข้อควรระวังก่อนเริ่มเขียนโค้ด

เขียนโค้ดให้มีคุณภาพ

1. ตรวจสอบ input เสมอ: อย่าลืมคิดถึงกรณีค่าติดลบ ค่าว่าง หรือค่าที่ไม่อยู่ในช่วงที่กำหนด
2. ตั้งชื่อตัวแปรให้สื่อความหมาย และจัดย่อหน้า (indentation) ให้อ่านง่าย
3. เขียน comment อธิบายเฉพาะจุดที่ซับซ้อน ไม่จำเป็นต้อง comment ทุกบรรทัด
4. เน้นความอ่านง่าย เพราะโค้ดที่อ่านง่ายจะแก้บั๊กและต่อยอดได้ง่ายกว่า

จะเลือกใช้ if-else หรือ switch-case ดี?

ใช้ if-else เมื่อ:

- ต้องตรวจสอบ "ช่วง" ของค่า (เช่น มากกว่า/น้อยกว่า) หรือหลายเงื่อนไขพร้อมกัน
- ต้องตรวจสอบความถูกต้องของ input หรือดักกรณีผิดพลาด
- ต้องใช้ AND/OR รวมกันหลายเงื่อนไข

ใช้ switch-case เมื่อ:

- ต้อง map ค่าที่แน่นอนไปยังผลลัพธ์ (เช่น เลขวัน → ชื่อวัน, ตัวเลือกเมนู)
- มีหลายกรณีที่เป็นค่าคงที่ตายตัว
- อยากรู้ว่าโค้ดอ่านง่ายกว่าการเขียน if-else ต่อกันยาว ๆ

ก่อนส่งงาน ลองเช็คตามนี้

1. รัน test cases ของทุก method ให้ครบ ไม่ใช่แค่เคสแรก
2. ลองใส่ edge case ด้วย เช่น ค่าติดลบ ค่า 0 หรือค่าที่นอกช่วงที่กำหนด
3. เทียบข้อความ output กับ Test Cases ในเอกสารที่ละตัวอักษร (ตัวพิมพ์เล็ก-ใหญ่ เว้นวรรค เครื่องหมายวรรคตอน ต้องตรงกันหมด)
4. ถ้าผลลัพธ์เป็น boolean ให้ใช้ "True" หรือ "False" ขึ้นต้นด้วยตัวพิมพ์ใหญ่ตามที่ตัวอย่างกำหนด



เริ่มต้นอย่างไร

1. เปิดโปรเจกต์นี้ใน Unity แล้วเปิดไฟล์ `Workshop.cs` และ `Assignment.cs` ด้วย code editor (เช่น VS Code, Visual Studio หรือ Notepad)
2. เริ่มทบทวน Example Methods ใน `Workshop.cs` ตามที่เรียนในชั่วโมงก่อน เพื่อให้คุ้นชินกับ if-else และ switch-case แบบง่าย ๆ (ส่วนนี้ไม่มีคะแนน ทำเพื่อความเข้าใจ)
3. เขียนโค้ดใน method ที่ต้องการ ด้วย logic ของโจทย์แต่ละข้อ
4. กด Play ใน Unity เพื่อรันและดูผลลัพธ์ใน Console แล้วเทียบกับตัวอย่างที่ให้ เป็น Test Cases
5. ทำ Level 1 ให้ผ่านครบก่อน แล้วค่อยไปทำ Level 2 ใน `Assignment.cs`



ข้อกำหนดการส่งงาน

- เขียน code, commit และ push branch "submission"
- เขียน logic ให้ครบทั้ง 11 methods ใน `Assignment.cs` (As01-As11)
- ผลลัพธ์ต้องตรงกับ test cases ทั้งหมดที่ระบุไว้ในเอกสารนี้
- โค้ดควรอ่านง่าย มี comment อธิบายจุดที่จำเป็น
- แนะนำให้ใช้ `Debug.Log()` สำหรับแสดงผล

ขอให้โชคดีกับ assignment ของคุณ! 🎮 🧑